Django Complete Notes with Practical Examples

Prepared for Students

June 22, 2026

Contents

1 Introduction to Django

Django is a high-level Python web framework that enables rapid development of secure and maintainable web applications.

1.1 Why Django?

- Fast Development
- Secure Framework
- Scalable Applications
- Built-in Admin Panel
- ORM Support
- Authentication System
- Large Community Support

1.2 Applications of Django

- E-Commerce Websites
- Social Media Platforms
- CRM Systems
- ERP Systems
- Learning Management Systems
- AI Web Applications

2 Installing Django

2.1 Check Python Version

```
python --version
```

2.2 Create Virtual Environment

```
python -m venv myenv
```

Activate Environment
Windows

```
myenv\Scripts\activate
```

Linux/Mac

```
source myenv/bin/activate
```

2.3 Install Django

```
pip install django
```

Verify Installation

```
django-admin --version
```

3 Creating First Django Project

3.1 Create Project

```
django-admin startproject myproject
```

3.2 Move into Project

```
cd myproject
```

3.3 Run Development Server

```
python manage.py runserver
```

Open Browser:

```
http://127.0.0.1:8000
```

4 Project Structure

```
myproject/
|
|-- manage.py
|-- myproject/
|
|-- settings.py
|-- urls.py
|-- wsgi.py
|-- asgi.py
```

4.1 Important Files

- manage.py
- settings.py
- urls.py
- wsgi.py
- asgi.py

5 Creating Django App

```
python manage.py startapp student
```

Folder Structure

```
student/
|
|-- admin.py
|-- models.py
|-- views.py
|-- apps.py
|-- tests.py
```

6 Django MVT Architecture

MVT stands for:

1. Model
2. View
3. Template

6.1 Model

Handles database operations.

6.2 View

Contains business logic.

6.3 Template

Handles UI.

7 URL Routing

myproject/urls.py

```
from django.contrib import admin
from django.urls import path
from student import views

urlpatterns = [
path('admin/', admin.site.urls),
path('', views.home)
]
```

8 Views

student/views.py

```
from django.http import HttpResponse

def home(request):
return HttpResponse("Welcome to Django")
```

9 Templates

Create Folder

```
student/
|
|-- templates/
|
|-- home.html
```

home.html

```
<!DOCTYPE html>

<html>
<head>
<title>Home</title>
</head>
<body>
<h1>Welcome to Django</h1>
</body>
</html>
```

views.py

```
from django.shortcuts import render

def home(request):
return render(request,'home.html')
```

10 Models

student/models.py

```
from django.db import models

class Student(models.Model):
name = models.CharField(max_length=100)
age = models.IntegerField()
email = models.EmailField()
```

```
```
def __str__(self):
    return self.name
```
```

11 Migrations

Create Migration

```
python manage.py makemigrations
```

Apply Migration

```
python manage.py migrate
```

12 Admin Panel

Create Super User

```
python manage.py createsuperuser
```

Register Model
admin.py

```
from django.contrib import admin
from .models import Student

admin.site.register(Student)
```

Run Server

```
python manage.py runserver
```

Admin URL

```
http://127.0.0.1:8000/admin
```

13 Forms

forms.py

```
from django import forms

class StudentForm(forms.Form):
name = forms.CharField(max_length=100)
age = forms.IntegerField()
```

14 Model Forms

```
from django.forms import ModelForm
from .models import Student

class StudentForm(ModelForm):

'''
class Meta:
    model = Student
    fields = "__all__"
'''
```

15 CRUD Application

15.1 Create Student

views.py

```
Student.objects.create(
name="Aftab",
age=22,
email="[aftab@gmail.com](mailto:aftab@gmail.com)"
)
```

15.2 Read Data

```
students = Student.objects.all()
```

15.3 Update Data

```
student = Student.objects.get(id=1)
student.age = 25
student.save()
```

15.4 Delete Data

```
student = Student.objects.get(id=1)
student.delete()
```

16 Authentication

16.1 User Registration

```
from django.contrib.auth.models import User

User.objects.create_user(
```

```
username='aftab',
password='1234'
)
```

16.2 Login

```
from django.contrib.auth import authenticate

user = authenticate(
username='aftab',
password='1234'
)
```

17 File Upload

models.py

```
image = models.ImageField(
upload_to='images/'
)
```

Install Pillow

```
pip install pillow
```

18 MySQL Configuration

Install Driver

```
pip install mysqlclient
```

settings.py

```
DATABASES = {
'default': {
'ENGINE':
'django.db.backends.mysql',
'NAME':'studentdb',
'USER':'root',
'PASSWORD':'root',
'HOST':'localhost',
'PORT':'3306'
}
}
```

19 Django REST Framework

Install

```
pip install djangorestframework
```

Serializer

```
from rest_framework import serializers
from .models import Student

class StudentSerializer(
serializers.ModelSerializer):

'''
class Meta:
    model = Student
    fields = "__all__"
'''
```

APIView

```
from rest_framework.views import APIView
from rest_framework.response import Response

class StudentAPI(APIView):

'''
def get(self, request):
    return Response({
        "message":"Success"
    })
'''
```

20 Deployment

Popular Platforms:

- Render
- Railway
- PythonAnywhere
- AWS
- Digital Ocean

21 Mini Projects

1. Student Management System
2. Employee Management System
3. Hospital Management System
4. Blog Website
5. Library Management System

6. E-Commerce Website
7. AI Resume Screening System
8. AI Chatbot using Django

22 Interview Questions

22.1 Basic

1. What is Django?
2. Explain MVT Architecture.
3. What is ORM?
4. Difference between Django and Flask?
5. What are migrations?

22.2 Intermediate

1. Explain Middleware.
2. Explain Class Based Views.
3. What is ModelForm?
4. Explain Authentication.
5. What is Django REST Framework?

23 Assignments

1. Create Student Registration System.
2. Build Login Page.
3. Create CRUD Application.
4. Build Library Management System.
5. Build Employee Management System.
6. Create REST API.
7. Upload Images using Django.
8. Connect MySQL Database.
9. Deploy Application.
10. Create Blog Website.

24 Career Roadmap

1. Python Fundamentals
2. OOP Concepts
3. Django Basics
4. Django ORM
5. REST APIs
6. Authentication
7. MySQL/PostgreSQL
8. Deployment
9. Docker
10. AWS
11. AI Integration
12. Production Projects

25 Conclusion

Django is one of the most powerful Python web frameworks. By mastering Django, ORM, REST APIs, Authentication, Deployment, and Real-world Projects, students can become Full Stack Python Developers and build enterprise-level applications.